
Optimized Modeling and Predictive Analysis of Complex Systems

Summary

Problem Restatement. In one or two sentences, state the **core question** and **constraints** in your own words; identify **decision variables**, **objectives**, and **feasible region**.

Uniqueness of the Modeling Approach. Briefly state the **model class** (e.g., differential equations, optimization, stochastic simulation) and **distinct assumptions** or **innovations** relative to existing methods.

Core Algorithm Logic. Outline the **solution pipeline**: inputs, main iterations or recurrences, and output metrics; name **software/algorithms** when relevant (e.g., Gurobi, Monte Carlo).

Most Impactful Quantitative Conclusions. Give concrete **numerical results** and **policy implications** (e.g., “cost reduced by 15.4%” or “service level above 98%”); avoid vague phrases. Optionally state **sensitivity** or **robustness** findings.

The four blocks above use bold terms and short paragraphs for modular scannability, with paragraph spacing for clarity.

Keywords: Mathematical Modeling; Sensitivity Analysis; AI Tools Disclosure.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Literature Review	3
1.3	Restatement of the Problem	3
2	Assumptions and Notations	3
2.1	General Assumptions	3
2.2	Notation Definition	3
3	Data Sources	4
3.1	Data Description and Task DNA Source	4
4	Model Construction and Implementation	4
4.1	Career Selection as a Modeled Decision	4
4.1.1	Modeling Rationale	4
4.1.2	Objectification of Career Feature Parameters Based on O*NET	6
4.1.3	Optimization Formulation	8
4.1.4	Revision and Final Configuration of Selected Careers	10
4.2	Model Selection for Occupational Evolution	10
4.2.1	Computable Definition of Future Evolution	11
4.2.2	Comparison of Candidate Modeling Approaches	11
4.2.3	Final Model Choice	11
4.3	Task-Aggregate Dynamic Model of Occupational Evolution	12
4.3.1	Task Aggregation and Occupational Dynamics	12
4.4	GenAI Capability-Dimension Scoring from Authoritative Reports: Scale Definition and Evidence Narrative	12
4.4.1	Scale Definition	12
4.4.2	Capability Evidence by Task Type (Narrative Justification)	13
4.5	Authoritative Evaluation Driven Relabeling of Task Impacts: From Keyword Tagging to Capability-Dimension Assignment	14
4.6	Aggregating Task DNA into Occupation-Level Shock Parameters: Software Developers	16
4.6.1	Top- N Availability and Aggregation	16
4.6.2	Interpretability Decomposition: Top- K Contribution Lower Bounds	16
4.6.3	Validity Check of the Scoring Rule: Avoiding Near-Constant s/c	16
4.6.4	Minimal Calibration: Category Defaults with Limited Manual Review	17
5	Mechanistic Power Check and Occupation-Level Aggregation After Authoritative Relabeling (Software Developer)	17
5.1	Occupation-Level Shock Aggregates	17
5.2	Diagnostics: Heterogeneity and Effective Diversity	17
5.3	Engineering Thresholds and Actions (Gatekeeping Before $E(t)$)	18

6	Consistency Check and Instantiating the Evolution Equation: (α, β, μ) for Software Developers	18
6.1	Aggregates and Consistency Check	18
6.2	Mechanistic Power (Heterogeneity) Check: No Score Collapse	19
6.3	Minimal Sensitivity for a Low-Confidence Task (Upper-Bound Argument)	19
6.4	Proceeding to Occupational Evolution After Fixing μ	19
7	Simulation Results and Sensitivity Analysis	19
8	Discussion: Strengths and Weaknesses	19
8.1	Strengths	19
8.2	Weaknesses	20
9	Conclusion	20
	Appendices	21
	Appendix A Python Implementation Code	21

1 Introduction

1.1 Background and Motivation

Describe the real-world context provided by the problem statement.

AI use disclosure. Our use of AI tools is disclosed in this paragraph (in-text) and in full in the **Report on Use of AI Tools** at the end of this paper.

1.2 Literature Review

Cite existing research to build model credibility.[4]

1.3 Restatement of the Problem

Define specific goals using your own academic language.

2 Assumptions and Notations

2.1 General Assumptions

List simplifying assumptions and their scientific justifications.

2.2 Notation Definition

We organize notations into logical groups corresponding to the modeling pipeline. All symbols are defined once here and referenced consistently in subsequent sections.

Table 1: Notations for Career Selection and Feature Space

Symbol	Description	Domain/Units
x	Career candidate	—
$f(x)$	Career feature vector ($digit(x)$, $physical(x)$, $iprisk(x)$)	$[0, 1]^3$
$digit(x)$	Digitizability (O*NET descriptor proxy, normalized)	$[0, 1]$
$physical(x)$	Physical anchoring (O*NET descriptor proxy, normalized)	$[0, 1]$
$iprisk(x)$	IP/attribution risk proxy (originality intensity)	$[0, 1]$
$D(x)$	Data availability score	$[0, 1]$
$\Delta(x_i, x_j)$	Mechanistic contrast distance $\ f(x_i) - f(x_j)\ _2$	$\mathbb{R}_{\geq 0}$
λ	Trade-off coefficient in career selection	Dimensionless

Table 2: Notations for O*NET Descriptor Normalization

Symbol	Description	Domain/Units
$v_m(x)$	Raw O*NET descriptor m rating for occupation x	$[0, 100]$
$z_m(x)$	Min-max normalized rating: $\frac{v_m(x) - \min_y v_m(y)}{\max_y v_m(y) - \min_y v_m(y)}$	$[0, 1]$

Table 3: Notations for the Task-Aggregate Evolution Model

Symbol	Description	Domain/Units
i	Occupation index	$\mathbb{Z}^+$
j	Task index	$\mathbb{Z}^+$
t	Time	Years
$E_i(t)$	Employment demand for occupation i at time t	Jobs
$T_i(t)$	Task structure of occupation i (task weight reallocation)	—
$A(t)$	GenAI adoption trajectory (scenario family: fast/medium/slow)	$[0, 1]$
g_i	Non-AI baseline growth/decay rate for occupation i	yr^{-1}

Table 4: Notations for Task Weights and Impact Parameters

Symbol	Description	Domain/Units
w_{ij}	Weight of task j in occupation i ; $\sum_j w_{ij} = 1$	$[0, 1]$
s_{ij}	Substitution intensity for task j in occupation i	$\{0.25, 0.5, 0.75\}$
c_{ij}	Augmentation intensity for task j in occupation i	$\{0.25, 0.5, 0.75\}$
α_i	Occupation-level substitution: $\sum_j w_{ij}s_{ij}$	$[0, 1]$
β_i	Occupation-level augmentation: $\sum_j w_{ij}c_{ij}$	$[0, 1]$
μ_i	Net shock for occupation i : $\beta_i - \alpha_i$	$[-1, 1]$

3 Data Sources

3.1 Data Description and Task DNA Source

We extract task statements and task ratings for SOC 15-1252.00 from the O*NET 30.1 text database, and we compute task weights from Importance (IM) and Frequency of Task (FT) ratings.[8]

4 Model Construction and Implementation

4.1 Career Selection as a Modeled Decision

A prerequisite of the problem requires selecting one career each from the STEM, Trade, and Arts sectors. Rather than treating this step as an ad hoc assumption, we explicitly model career selection as an optimization problem. This ensures that subsequent analyses of AI-driven impacts and educational recommendations are grounded in a reproducible and data-informed framework.

4.1.1 Modeling Rationale

The core objective of career selection is not representativeness of all occupations, but *explanatory contrast*. Specifically, the selected careers should differ substantially in how generative artificial intelligence (GenAI) interacts with their task structures, while remaining empirically tractable and policy-relevant.

Based on the problem statement and data availability constraints, we identify three dominant dimensions that govern heterogeneity in AI impact across occupations:

Table 5: Notations for Task DNA Fields

Symbol	Description	Domain/Units
Task_j	Task statement text (O*NET Task Statements)	Text
IM_j	Task importance rating	[0, 100]
FR_j	Task frequency rating (or equivalent scale)	[0, 100]
$\tilde{w}_j$	Unnormalized weight: $\text{IM}_j \cdot \text{FR}_j$ (or IM_j if FR missing)	$\mathbb{R}_{\geq 0}$
w_j	Normalized task weight for Software Developer ($= w_{ij}$)	[0, 1]
N	Number of usable tasks (Top- N ; currently $N = 17$ for Software Dev)	$\mathbb{Z}^+$
SOC	O*NET-SOC code (Software Developer: 15-1252.00)	Code

Table 6: Notations for Authoritative Relabeling

Symbol	Description	Domain/Units
dim_j	Capability dimension set mapped to task j (1–2 dimensions)	Set
d	Capability dimension index (e.g., code generation, debugging)	—
S_d	Substitution tier for dimension d	{0.25, 0.5, 0.75}
C_d	Augmentation tier for dimension d	{0.25, 0.5, 0.75}
$\mathcal{G}_{LMH}$	Discrete tier set {0.25, 0.5, 0.75}	—
$\text{sim}(j, d)$	Semantic similarity score between task j and dimension d	[0, 1]
τ	Semantic matching threshold	[0, 1]
Override	Human override list (few high-weight ambiguous tasks)	List
AnomalyReport	Anomaly report (low similarity / conflict / unmatched)	Report

- **Digitizability:** the extent to which core tasks can be represented, automated, or augmented digitally;
- **Physical anchoring:** the degree to which task execution requires on-site physical presence or manual interaction with the environment;
- **Intellectual property and attribution risk:** the sensitivity of value creation to authorship, originality, and ownership.

Each candidate career is represented by a normalized feature vector

$$f(x) = (\text{digit}(x), \text{physical}(x), \text{iprisk}(x)) \in [0, 1]^3, \quad (1)$$

where higher values indicate stronger presence of the corresponding characteristic.

In addition to mechanistic contrast, the problem explicitly emphasizes data-informed reasoning. Therefore, we introduce a data availability score $D(x)$ that reflects whether reliable data sources exist for employment trends, task descriptions, compensation, and educational pathways associated with career x . These data sources include occupational databases [8], post-secondary employment outcomes [10], educational institutional data [6], and occupational classification crosswalks [7].

Table 7: Notations for Mechanistic Power Diagnostics (Software Developer Instance)

Symbol	Description	Domain/Units
α	Substitution aggregate: $\sum_{j=1}^N w_j s_j$ (preview: 0.379113)	$[0, 1]$
β	Augmentation aggregate: $\sum_{j=1}^N w_j c_j$ (preview: 0.629112)	$[0, 1]$
μ	Net shock: $\beta - \alpha$ (by definition: 0.249999)	$[-1, 1]$
$\text{Var}_w(s)$	Weighted variance of s : $\sum w_j (s_j - \alpha)^2$ (preview: 0.015608)	$\mathbb{R}_{\geq 0}$
$\text{Var}_w(c)$	Weighted variance of c : $\sum w_j (c_j - \beta)^2$ (preview: 0.015608)	$\mathbb{R}_{\geq 0}$
$h_s(g)$	Tier weight mass for s : $\sum_{j: s_j=g} w_j$	$[0, 1]$
$h_c(g)$	Tier weight mass for c : $\sum_{j: c_j=g} w_j$	$[0, 1]$
$m(d)$	Dimension weight mass: $\sum_j w_j \pi_{jd}$	$[0, 1]$

Table 8: Engineering Thresholds for Mechanistic Power Gatekeeping

Symbol	Description	Default
ε_s	Minimum required $\text{Var}_w(s)$	0.005
ε_c	Minimum required $\text{Var}_w(c)$	0.005
η_s	Maximum allowed $\max_g h_s(g)$ (tier concentration)	0.70
η_d	Maximum allowed $\max_d m(d)$ (dimension concentration)	0.60

4.1.2 Objectification of Career Feature Parameters Based on O*NET

To avoid subjective assignment of the career feature parameters $\text{digit}(x)$, $\text{physical}(x)$, and $\text{iprisk}(x)$, we adopt the **O*NET** occupational database maintained by the U.S. Department of Labor as a unified data source, with detailed data provenance summarized in Section 3. O*NET provides a large number of *numerical scale descriptors* based on expert surveys and practitioner feedback for each occupation, with most indicators naturally given on a continuous scale of 0–100, suitable for direct use in quantitative modeling.

Parameter Construction Logic We do not attempt to find the “unique correct” indicator for each dimension, but rather follow the *shortest defensible path* principle: for each feature axis, we select a small number of semantically directly corresponding and downloadable O*NET descriptors and use them as proxy variables for that axis.

- **Digitizability** $\text{digit}(x)$ This dimension characterizes the extent to which occupational tasks rely on computers and electronic information flows. We select work activity and work context descriptors from O*NET that are directly related to “working with computers” and “frequency of electronic communication”. The higher the value, the more the occupation is centered on digital media.
- **Physical anchoring** $\text{physical}(x)$ This dimension characterizes the degree to which an occupation depends on on-site operations, manual control, and physical posture constraints. We use work context descriptors that reflect “time spent performing hands-on activities” and

Table 9: Notations for Sensitivity Analysis

Symbol	Description	Domain/Units
w_{lc}	Weight of a low-confidence task (example: 0.05434)	[0, 1]
$\Delta_{\max, \text{score}}$	Maximum single-task tier swing: $0.75 - 0.25 = 0.50$	Dimensionless
$\Delta_{\max}(\alpha)$	Worst-case perturbation to α : $w_{lc} \cdot \Delta_{\max, \text{score}}$	[0, 1]
$\Delta_{\max}(\beta)$	Worst-case perturbation to β : $w_{lc} \cdot \Delta_{\max, \text{score}}$	[0, 1]

Table 10: Notations for Occupational Evolution Dynamics

Symbol	Description	Domain/Units
g	Non-AI baseline growth rate (trend estimate or set to 0)	yr ⁻¹
$E(t)/E(0)$	Relative demand index: $\exp\left(\int_0^t (g + A(\tau)\mu) d\tau\right)$	Dimensionless
mu_definition	Metadata: must record $\mu = \beta - \alpha$ for consistency audit	—

“physical posture burdens such as kneeling, squatting, or bending”, to quantify whether the occupation requires physical presence and physical operations.

- **Intellectual property / attribution risk $iprisk(x)$ (proxy indicator)** This dimension does not directly measure the probability of legal litigation, but rather serves as a *proxy indicator* to characterize the dependence of occupational output on original expression and creative contribution. We select ability and work activity descriptors related to “Originality” and “Thinking Creatively” from O*NET. The higher the intensity of original expression, the more likely it is to trigger copyright, attribution, or style attribution disputes when generative models intervene.

Table 11: Data Sources and Variables for Task DNA

Variable/Data	Description	Source
O*NET Task State-ments	Task text and identifiers for SOC 15-1252.00	O*NET database [8] 30.1
O*NET Task Ratings (IM, FT)	Importance and Frequency ratings used to compute task weights	O*NET database [8] 30.1
Task DNA CSV	Processed task table with IM, FR, weight, and rule-based s/c	Derived from O*NET 30.1 [8]

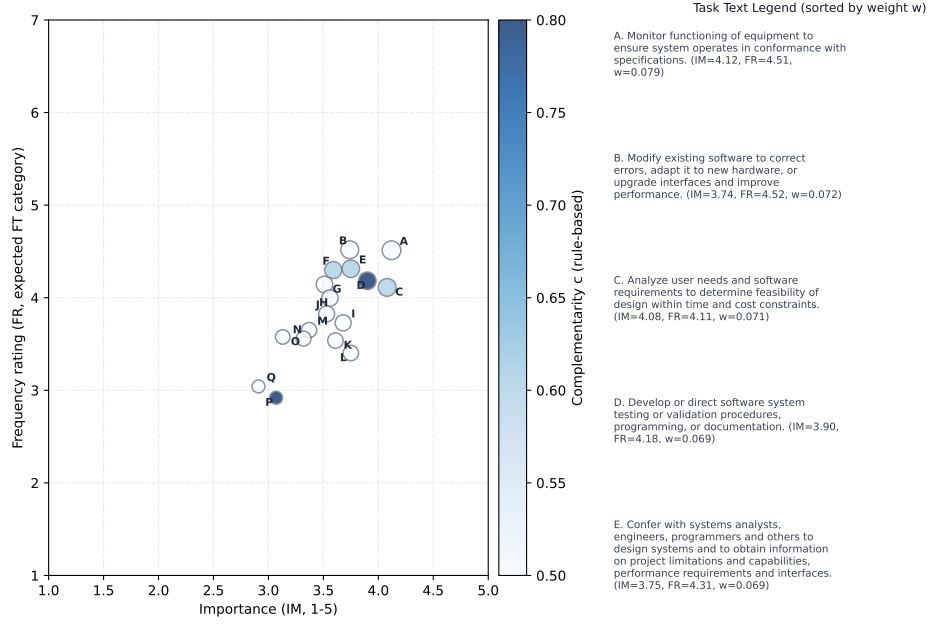


Figure 1: Task-level IM–FR exposure for Software Developers (SOC 15-1252.00). Bubble size encodes normalized task weight w , color encodes rule-based complementarity c , and edge thickness reflects rule-based substitution s . Letter tags map to the task text legend panel.

Normalization and Reproducibility For each occupation x , let $v_m(x)$ denote its raw rating (0–100) under O*NET descriptor m . We apply min-max normalization across the entire occupation set:

$$z_m(x) = \frac{v_m(x) - \min_y v_m(y)}{\max_y v_m(y) - \min_y v_m(y)} \in [0, 1], \quad (2)$$

and take the equally weighted average of multiple descriptors under the same feature axis to obtain $digit(x)$, $physical(x)$, and $iprisk(x)$. This process is entirely based on publicly available data and explicit rules, ensuring the reproducibility and auditability of parameter values.

We visualize the three-dimensional heterogeneity and its 2D projections to validate that the selected careers occupy distinct regions in the feature space (Fig. 2–3).

4.1.3 Optimization Formulation

For a triplet of careers (x_1, x_2, x_3) drawn respectively from STEM, Trade, and Arts categories, we define the pairwise contrast between two careers as the Euclidean distance between their feature vectors:

$$\Delta(x_i, x_j) = \|f(x_i) - f(x_j)\|_2. \quad (3)$$

The overall selection objective is then formulated as

$$\max_{x_1 \in \text{STEM}, x_2 \in \text{Trade}, x_3 \in \text{Arts}} \left(\sum_{i < j} \Delta(x_i, x_j) + \lambda \sum_{k=1}^3 D(x_k) \right), \quad (4)$$

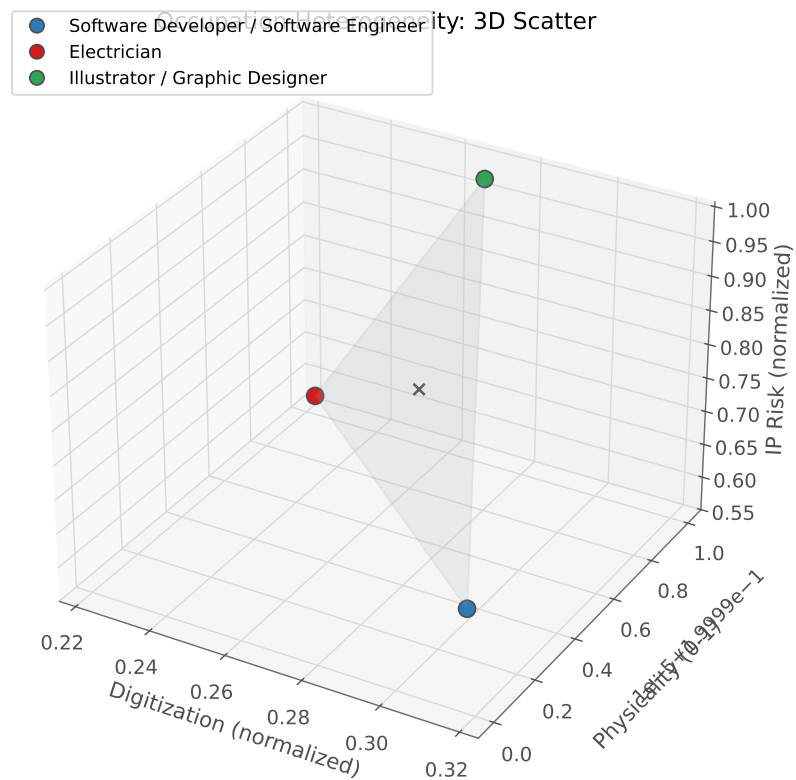


Figure 2: 3D heterogeneity of the selected STEM, Trade, and Arts occupations based on O*NET descriptors (global normalization).

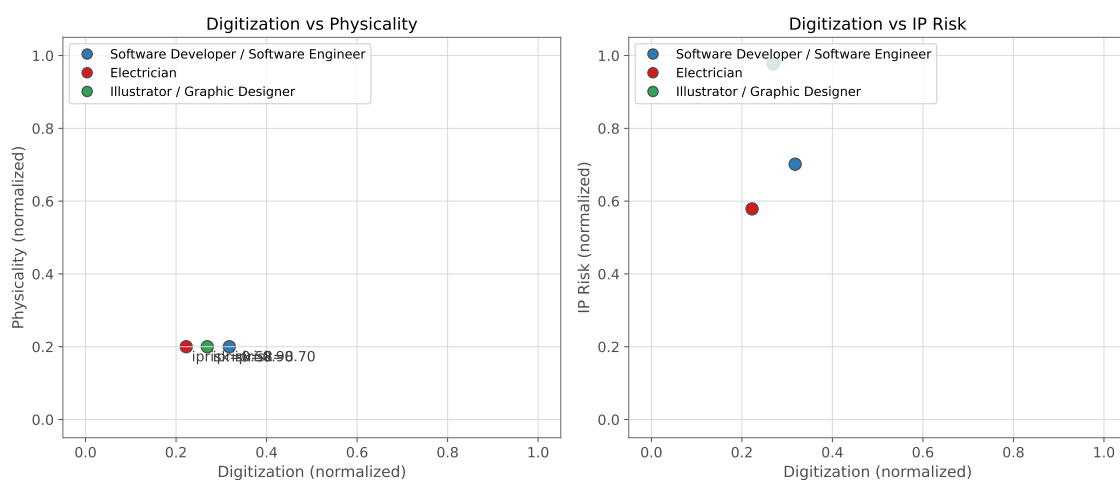


Figure 3: 2D projections of the heterogeneity space; the annotations report the remaining axis value for interpretability (global normalization).

where λ balances explanatory contrast against empirical feasibility.

This formulation does not claim global optimality across all possible occupations. Instead, it yields a deliberately constructed high-contrast career set that maximizes analytical clarity under realistic data and time constraints.

4.1.4 Revision and Final Configuration of Selected Careers

In the initial scheme, we selected Structural/Civil Engineer as the STEM occupation. However, upon further examination of the distribution of the three occupations across the “task structure” and “physical constraint” dimensions, we found that this choice exhibits some similarity with the Trade occupation (Electrician) along the “physical anchoring strength” axis, potentially weakening the contrast among the three occupations in the mechanism space.

To maximize task structure heterogeneity and enhance the identifiability of subsequent GenAI impact mechanisms, we revise the STEM endpoint by replacing it with **Software Developer / Software Engineer**.

This revision does not alter the structure of the original selection model, but merely adjusts the position of the occupational endpoint in the feature space: software developers possess extremely high digitizability and extremely low physical anchoring, thereby significantly increasing their distance from electricians in the feature vector $f(x)$ and improving the value of the overall contrast objective function.

It must be emphasized that we do not assume that the software development occupation will be simply “replaced” by GenAI. On the contrary, this occupation is more likely to undergo **task structure reorganization**: the proportion of coding and boilerplate implementation decreases, while the proportion of tasks such as requirement clarification, system architecture design, verification testing, and security and compliance responsibilities increases. It is precisely this shift from “execution-oriented tasks” to “verification and responsibility-oriented tasks” that makes software development an ideal STEM endpoint for studying GenAI impact mechanisms.

After completing this revision, the final configuration of the three selected occupations is as follows:

- **STEM: Software Developer / Software Engineer.** High digitizability, low physical anchoring; GenAI can directly intervene in core tasks, but primarily manifests as task redistribution rather than occupational extinction.
- **Trade: Electrician.** High physical anchoring, strong on-site constraints; AI primarily serves as a planning, diagnostic, and auxiliary tool, difficult to replace core operational tasks.
- **Arts: Commercial Illustrator / Graphic Designer.** High intellectual property and attribution sensitivity; GenAI directly affects output forms and value pricing mechanisms.

4.2 Model Selection for Occupational Evolution

The second core requirement of the problem is to construct a *data-informed* model of future occupational evolution. Before introducing formal equations, we first define what “future evolution”

means in computable terms and then systematically evaluate alternative modeling approaches, so that model choice itself is not an implicit assumption.

4.2.1 Computable Definition of Future Evolution

In this paper, the future evolution of occupation i is characterized by two primary outputs:

- **Occupational demand:** the time evolution of job demand $E_i(t)$ over a 10-year horizon, under fast, medium, and slow GenAI adoption scenarios;
- **Task structure:** the reallocation of task weights $T_i(t)$ within the occupation, indicating which tasks expand, contract, or are reorganized.

This definition ensures that occupational-level outcomes are derived from task-level mechanisms rather than imposed directly.

4.2.2 Comparison of Candidate Modeling Approaches

We consider three representative approaches for modeling occupational evolution:

Approach A: Task-aggregate impact model This approach treats an occupation as a weighted bundle of tasks. GenAI-driven substitution and augmentation effects are specified at the task level and then aggregated to the occupational level using task weights. Most parameters can be constructed from O*NET task data [8] and publicly available AI exposure rubrics [4], providing transparency and reproducibility.

Approach B: Job-posting time-series models This approach infers occupational change from trends in AI-related keywords and skill requirements in job postings. While closely tied to market signals, it suffers from identification challenges and high data-processing overhead, making it unsuitable as the main modeling backbone.

Approach C: Agent-based simulations Agent-based models simulate strategic interactions among firms, workers, and educational institutions. Despite their narrative richness, they require extensive calibration and introduce substantial model complexity, which conflicts with transparency requirements under strict page limits.

4.2.3 Final Model Choice

Based on the above comparison, we adopt the task-aggregate impact model (Approach A) as our primary framework. This choice is motivated by:

- **Mechanistic clarity:** occupational change emerges explicitly from task-level effects;
- **Data alignment:** key parameters are grounded in publicly available occupational data [8, 10];
- **Controlled complexity:** the model remains interpretable and suitable for scenario and sensitivity analysis;

- **Transferability:** the framework can be readily extended to other occupations or educational pathways [7].

Approach B is reserved for potential validation or supplementary analysis, while Approach C is excluded from the main analysis due to calibration and transparency concerns.

4.3 Task-Aggregate Dynamic Model of Occupational Evolution

Having justified the task-aggregate approach, we now present its mathematical formulation. We adopt a minimal task-to-occupation framework: GenAI adoption $A(t)$ enters as an exogenous driver, substitution and augmentation are specified at the task level, and occupation-level net impact is derived by task-weight aggregation, yielding a dynamic trajectory for occupational demand $E_i(t)$.

4.3.1 Task Aggregation and Occupational Dynamics

Let occupation i consist of tasks $j = 1, \dots, J_i$ with weights w_{ij} such that

$$\sum_{j=1}^{J_i} w_{ij} = 1. \quad (5)$$

For each task j , let $s_{ij} \in [0, 1]$ denote substitution intensity and $c_{ij} \in [0, 1]$ denote augmentation (complementarity) intensity. Define occupation-level aggregate effects as

$$\alpha_i = \sum_{j=1}^{J_i} w_{ij} s_{ij}, \quad \beta_i = \sum_{j=1}^{J_i} w_{ij} c_{ij}. \quad (6)$$

Let $E_i(t)$ be employment demand for occupation i at time t , and let g_i capture baseline growth/decay from non-AI factors (either estimated from public trends or set to 0 under scenario analysis). Given a scenario family of adoption trajectories $A(t) \in [0, 1]$ (fast/medium/slow), we model occupational evolution as

$$\frac{dE_i(t)}{dt} = E_i(t) \left(g_i + A(t) (\beta_i - \alpha_i) \right). \quad (7)$$

This formulation ensures that occupational outcomes are derived from task-level mechanisms, supporting interpretability, reproducibility, and data-informed reasoning.

4.4 GenAI Capability-Dimension Scoring from Authoritative Reports: Scale Definition and Evidence Narrative

To replace the subjective “keyword tagging” method, we adopt capability ratings derived from authoritative evaluations and official technical reports. These ratings quantify GenAI **substitution** and **augmentation** capabilities across major software-development task dimensions, providing auditable grounds for mapping Software Developer Task DNA to (s, c) inputs.

4.4.1 Scale Definition

We use three levels (Low/Medium/High) to represent GenAI capability, encoded as 0.25, 0.50, and 0.75:

- **Low (0.25):** GenAI has limited autonomous ability on this task type and cannot reliably deliver satisfactory results without human intervention; it can replace only a small portion of work, but may still provide assistance under human supervision.
- **Medium (0.50):** GenAI performs at a moderate level, completing parts of the task or offering useful recommendations, yet still requires human review and collaboration; it cannot independently replace human execution.
- **High (0.75):** GenAI performs strongly and is close to professional human quality for many subtasks; under proper constraints and verification, it can handle a large share of the workflow and deliver significant augmentation, though it remains imperfect.

Note: the values 0.25/0.50/0.75 are coarse modeling encodings rather than precise measurements. All levels are grounded in publicly available reports and comparative experiments released within the last two years.

4.4.2 Capability Evidence by Task Type (Narrative Justification)

Code Generation **Substitution: High (0.75); Augmentation: High (0.75).** GenAI has achieved strong performance in automated code generation. OpenAI GPT-4 reaches 67.0% pass@1 on HumanEval, far above GPT-3.5 at 48.1%; Anthropic Claude 2 reaches 71.2% on the same benchmark. This indicates that leading models solve more than half of typical coding problems correctly. Google CEO Sundar Pichai stated that roughly 25% of new code at Google is generated by AI tools, reflecting substantial productivity gains: given intent or comments, models can produce large volumes of draft code and accelerate development. For simple modules, models can partially substitute manual coding. Nevertheless, outputs still require human review and testing to mitigate logic errors and security issues. Overall, coding/implementation is among the most substitution-prone and augmentation-rich domains for GenAI.

Testing and Verification **Substitution: Medium (0.50); Augmentation: High (0.75).** GenAI shows moderate but improving capability for generating test cases and supporting verification. Studies report that LLM-generated unit tests can achieve coverage comparable to advanced automated tools and are often more readable. For instance, IBM Research experiments guided by static analysis found that GPT-based systems can generate Java/Python unit tests with competitive coverage and defect detection, while producing developer-friendly tests. This supports strong augmentation in routine function-level verification. However, generated tests may target irrelevant scenarios or weak assertions, requiring human review; thus full substitution remains difficult. In practice, GenAI can substantially reduce manual test-writing and speed up result checking, while humans validate test correctness.

Debugging and Bug Fixing **Substitution: Low (0.25); Augmentation: Medium (0.50).** End-to-end debugging remains a weak area: autonomous success rates are relatively low, but assistant value is tangible. A 2025 Microsoft Research study reported that even with debugger tool access, the best model (Anthropic Claude 3.7 Sonnet) solved only 48.4% of 300 curated debugging tasks on average, while others were around 30%. This suggests GenAI cannot fully replace human debugging. Limitations include ineffective use of debugging signals, poor strategy selection, and

lack of stepwise reasoning traces. Therefore substitution is rated low. Meanwhile, augmentation is evident: with logs or exception traces, models can propose useful fixes; “self-debugging” via few-shot demonstrations improves success by about 2%–12% in some settings. In practice, GenAI functions more as an assistant than an independent mechanic.

System Design / Architecture **Substitution: Low (0.25); Augmentation: Medium (0.50).** Architecture and system design demand global reasoning and responsible trade-offs; GenAI currently provides limited support. While no single authoritative benchmark directly measures system-design ability, models often produce plausible but incomplete plans, miss critical constraints, or rely on incorrect assumptions. Because system design lacks a single correct answer and requires accountable decision-making, autonomous substitution is rated low. Still, augmentation can be meaningful: models can draft design documents, summarize patterns, and brainstorm alternatives as a starting point for architects, improving discussion efficiency.

Requirements Analysis **Substitution: Low (0.25); Augmentation: Medium (0.50).** Requirements work requires deep domain context and ambiguity resolution. GenAI can assist with summarizing and structuring text, but cannot replace analysts. Although dedicated evaluations are limited, related tasks such as NL-to-formal specifications provide evidence: on Spider (NL-to-SQL), even with enhancement techniques, GPT-4 gains are only about 2%–3%, and complex queries remain challenging. This implies unreliable understanding for complex, ambiguous requirements, so substitution is low. However, augmentation is moderate: models can summarize lengthy documents, generate user-story templates, propose clarification questions, and accelerate organization of requirements under human supervision.

User Communication and Documentation **Substitution: Medium (0.50); Augmentation: High (0.75).** GenAI excels in language generation, strongly augmenting user communication and technical documentation. Models such as GPT-4 and Claude 2 can produce coherent long-form drafts and adhere to structured formats (e.g., Markdown/JSON), enabling rapid drafting of API docs, manuals, and user responses. Many routine user inquiries can be answered fluently and consistently. Yet accuracy and hallucination risks remain, especially for system-specific details, so fully replacing human communication is still risky; substitution is rated medium while augmentation is rated high.

4.5 Authoritative Evaluation Driven Relabeling of Task Impacts: From Keyword Tagging to Capability-Dimension Assignment

To improve the defensibility and reproducibility of task impact parameters (s_{ij}, c_{ij}) , we discard subjective keyword tagging and adopt an “**authoritative evaluations** → **capability dimensions** → **semantic matching** → **piecewise assignment**” relabeling workflow. The key idea is to extract substitution/augmentation levels for a set of *capability dimensions* (e.g., code generation, debugging, testing/verification, requirements analysis, system design, documentation/communication) from official evaluation reports, map each Software Developer task to 1–2 capability dimensions, and then assign (s_j, c_j) on a discrete grid.

Capability mapping and traceability requirement For each task j , define its mapped capability set dim_j (typically 1–2 dimensions for interpretability). Task-level substitution and augmentation are

Table 12: GenAI capability-dimension rubric from authoritative evaluation reports (for mapping Software Developer tasks to *s/c*)

Task Type	Substitution Score	Augmentation Score	Main evidence and support
Code generation	High (0.75)	High (0.75)	GPT-4 achieves 67.0% pass@1 on HumanEval [9]. Claude 2 reaches 71.2% [2]. Google reports about 25% of new code is generated by AI [11].
Testing and validation	Medium (0.50)	High (0.75)	LLM-generated unit tests can reach competitive coverage and readability. Studies suggest higher-quality tests when guided by static analysis [1]. Human review is still required, so substitution remains moderate.
Debugging and bug fixing	Low (0.25)	Medium (0.50)	Microsoft Research reports that even with debugging tools, end-to-end debugging success is below 50% (best model 48.4%) [11]. Prompted repair suggestions help, and “self-debugging” improves accuracy by up to 12 percentage points [3].
System design	Low (0.25)	Medium (0.50)	Complex architecture requires integrated reasoning; current GenAI often yields partial solutions [9]. Direct quantitative benchmarks are limited, but models can assist with drafts and documentation to speed design discussions [5].
Requirements analysis	Low (0.25)	Medium (0.50)	Model gains on NL→spec tasks are limited (e.g., 2–3% on Spider) [3]. However, summarization and question-generation can improve elicitation and analysis efficiency [9].
User communication and documentation	Medium (0.50)	High (0.75)	Models such as Claude 2 can generate long-form structured documentation and answer FAQs efficiently [2]. Accuracy remains a concern; critical content still needs human verification [9].

assigned on a discrete scale:

$$s_j, c_j \in \{0.25, 0.5, 0.75\}. \quad (8)$$

To meet the "data-informed" audit standard, each task must carry a **traceable chain**: (i) mapped capability dimension(s) dim_j , (ii) at least one authoritative source link, and (iii) a brief explanation of the task-to-dimension matching logic. This ensures (s_j, c_j) are reviewable and not based on researcher intuition.

Semantic thresholds, anomaly reporting, and limited human overrides Because semantic matching can incur both false negatives and false positives, the matching threshold is treated as a controllable decision parameter: too high may miss valid matches; too low may introduce mis-matches. Therefore, relabeling outputs must include similarity scores and an anomaly report (e.g., low-similarity matches, conflicting multi-dimension assignments), and allow a small human override list for a few high-weight tasks with ambiguous semantics. This preserves overall reproducibility while preventing a small number of mislabels from biasing the occupation-level aggregates (α_i, β_i) .

4.6 Aggregating Task DNA into Occupation-Level Shock Parameters: Software Developers

This section aggregates Software Developer Task DNA from task-level inputs (w_{ij}, s_{ij}, c_{ij}) into occupation-level shock parameters $(\alpha_i, \beta_i, \mu_i)$, which will feed the evolution equation $\frac{dE_i(t)}{dt} = E_i(t)(g_i + A(t)(\beta_i - \alpha_i))$.

4.6.1 Top- N Availability and Aggregation

In practice, O*NET task ratings may be missing for some tasks. We therefore adopt a "Top- N_i usable tasks" strategy: for occupation i , we keep only tasks with sufficient ratings to construct weights, and renormalize so that $\sum_{j=1}^{N_i} w_{ij} = 1$. Occupation-level substitution pressure, augmentation gain, and net direction are defined as

$$\alpha_i = \sum_{j=1}^{N_i} w_{ij} s_{ij}, \quad \beta_i = \sum_{j=1}^{N_i} w_{ij} c_{ij}, \quad \mu_i = \beta_i - \alpha_i. \quad (9)$$

4.6.2 Interpretability Decomposition: Top- K Contribution Lower Bounds

For interpretability, we decompose contributions from the top-weight tasks. Define partial sums

$$\alpha_i^{(K)} = \sum_{j \in \text{Top-}K} w_{ij} s_{ij}, \quad \beta_i^{(K)} = \sum_{j \in \text{Top-}K} w_{ij} c_{ij}. \quad (10)$$

For the currently available Top-5 sample, $\sum_{j \in \text{Top-}5} w_{ij} = 0.360897$, with partial contributions $\alpha_i^{(5)} = 0.1804485$ and $\beta_i^{(5)} = 0.2153186$, indicating that high-weight tasks (often testing/maintenance/requirements/collaboration) will dominate the aggregate shocks.

4.6.3 Validity Check of the Scoring Rule: Avoiding Near-Constant s/c

If many tasks share the same discrete score (e.g., 0.50), then α_i and β_i become weakly discriminative, undermining mechanistic explanation. We therefore check weight-mass concentration over the

discrete grid $\mathcal{G} = \{0, 0.25, 0.5, 0.75, 1\}$:

$$p_s(g) = \sum_{j: s_{ij}=g} w_{ij}, \quad p_c(g) = \sum_{j: c_{ij}=g} w_{ij}, \quad g \in \mathcal{G}, \quad (11)$$

using concentration and normalized entropy:

$$C_s = \max_{g \in \mathcal{G}} p_s(g), \quad C_c = \max_{g \in \mathcal{G}} p_c(g), \quad (12)$$

$$H_s = -\frac{\sum_{g \in \mathcal{G}} p_s(g) \ln p_s(g)}{\ln |\mathcal{G}|}, \quad H_c = -\frac{\sum_{g \in \mathcal{G}} p_c(g) \ln p_c(g)}{\ln |\mathcal{G}|}. \quad (13)$$

High C_s or low H_s flags score compression and triggers a minimal calibration.

4.6.4 Minimal Calibration: Category Defaults with Limited Manual Review

To avoid re-collecting data, we calibrate only s/c via a small set of task categories while keeping the task list and weights w_{ij} unchanged. Let $\kappa(j)$ map each task to a coarse category (e.g., requirements/stakeholders, coding/implementation, testing/validation/security, monitoring/operations) with a default (s, c) pair. We then manually review only a few top-weight tasks and record brief evidence notes, improving identifiability of (α_i, β_i) while preserving reproducibility and transparency.

5 Mechanistic Power Check and Occupation-Level Aggregation After Authoritative Relabeling (Software Developer)

After upgrading (s_j, c_j) from subjective keyword tagging to an "authoritative evaluations $\rightarrow$ capability dimensions $\rightarrow$ semantic matching $\rightarrow$ piecewise assignment" pipeline, we must validate **mechanistic power**: task-level heterogeneity in (s_j, c_j) should be sufficient, and task weights w_j should not collapse (α, β) into a single default-driven outcome. Otherwise, even authoritative sources would yield weak identifiability in the evolution model.

5.1 Occupation-Level Shock Aggregates

For the usable task set (size N , allowing $N < 20$) renormalized such that $\sum_{j=1}^N w_j = 1$, we define

$$\alpha = \sum_{j=1}^N w_j s_j, \quad \beta = \sum_{j=1}^N w_j c_j, \quad \mu = \beta - \alpha. \quad (14)$$

5.2 Diagnostics: Heterogeneity and Effective Diversity

(1) Weighted variance (heterogeneity)

$$\text{Var}_w(s) = \sum_{j=1}^N w_j (s_j - \alpha)^2, \quad \text{Var}_w(c) = \sum_{j=1}^N w_j (c_j - \beta)^2. \quad (15)$$

Low variance indicates compressed scoring or overly convergent mapping rules, weakening identifiability.

(2) Weighted tier histograms (avoid single-tier dominance) Let $\mathcal{G}_{LMH} = \{0.25, 0.5, 0.75\}$. Define tier weight mass

$$h_s(g) = \sum_{j: s_j=g} w_j, \quad h_c(g) = \sum_{j: c_j=g} w_j, \quad g \in \mathcal{G}_{LMH}. \quad (16)$$

Large $\max_g h_s(g)$ or $\max_g h_c(g)$ flags default-driven collapse.

(3) Weight mass by capability dimension (avoid weight concentration) For each task j , let dim_j be its mapped dimension set (1–2 dimensions). If multi-mapped, let π_{jd} allocate task j to dimension d with $\sum_{d \in \text{dim}_j} \pi_{jd} = 1$. Dimension weight mass is

$$m(d) = \sum_{j=1}^N w_j \pi_{jd}. \quad (17)$$

High $\max_d m(d)$ indicates low effective diversity despite broad nominal coverage.

5.3 Engineering Thresholds and Actions (Gatekeeping Before $E(t)$)

We use engineering thresholds as a gate to decide whether to proceed to scenario evolution:

$$\text{Var}_w(s) \geq \varepsilon_s, \quad \text{Var}_w(c) \geq \varepsilon_c, \quad \max_g h_s(g) \leq \eta_s, \quad \max_d m(d) \leq \eta_d, \quad (18)$$

with defaults $\varepsilon_s = \varepsilon_c = 0.005$, $\eta_s = 0.70$, and $\eta_d = 0.60$. Violations imply high mechanism risk, requiring refinement of tiering or the dimension-to- (s, c) mapping (without changing w_j) before advancing to scenario-driven $E(t)$ dynamics.

6 Consistency Check and Instantiating the Evolution Equation: (α, β, μ) for Software Developers

After relabeling task impacts via "authoritative evaluations $\rightarrow$ capability dimensions $\rightarrow$ semantic matching $\rightarrow$ discrete tiers (s, c) ," we aggregate Task DNA back to occupation-level shock parameters (α, β, μ) and conduct a consistency check first, preventing downstream dynamics from relying on an incorrectly implemented net-shock term.

6.1 Aggregates and Consistency Check

By definition,

$$\mu = \beta - \alpha. \quad (19)$$

Given the preview values $\alpha = 0.379113$ and $\beta = 0.629112$, we must have

$$\mu = 0.629112 - 0.379113 = 0.249999. \quad (20)$$

However, the preview JSON reports $\mu = 0.504112$, which contradicts the model definition. Therefore, before using μ as an input to $E(t)$, the aggregation output must explicitly state `mu_definition` and correct the implementation; otherwise, the evolution inference becomes systematically biased.

6.2 Mechanistic Power (Heterogeneity) Check: No Score Collapse

Using weighted variances

$$\text{Var}_w(s) = \sum_{j=1}^N w_j (s_j - \alpha)^2, \quad \text{Var}_w(c) = \sum_{j=1}^N w_j (c_j - \beta)^2, \quad (21)$$

the preview reports $\text{Var}_w(s) = \text{Var}_w(c) = 0.015608$, exceeding an engineering threshold (e.g., 0.005). This indicates that the relabeled (s, c) retain sufficient heterogeneity; mechanistic identifiability is not collapsed.

6.3 Minimal Sensitivity for a Low-Confidence Task (Upper-Bound Argument)

For a low-confidence match (e.g., task_id=21666 with weight $w = 0.05434$), robustness can be justified with a worst-case bound. Since tiers are $\{0.25, 0.5, 0.75\}$, the maximum per-task tier swing is 0.50 ($0.25 \leftrightarrow 0.75$), hence

$$\Delta_{\max}(\alpha) \leq w \cdot 0.50 = 0.05434 \times 0.50 = 0.02717, \quad (22)$$

and similarly $\Delta_{\max}(\beta) \leq 0.02717$. Two lightweight scenarios (dropping the task; remapping to the second-best dimension) suffice for a defensible sensitivity statement.

6.4 Proceeding to Occupational Evolution After Fixing μ

Once μ is corrected, the evolution equation becomes

$$\frac{dE(t)}{dt} = E(t)(g + A(t)\mu), \quad (23)$$

where $A(t)$ is the adoption scenario (fast/medium/slow) and g is the non-AI baseline (public trend estimate or set to 0 under a relative-index convention). The relative scale has the closed-form expression

$$\frac{E(t)}{E(t_0)} = \exp \left(\int_{t_0}^t (g + A(\tau)\mu) d\tau \right). \quad (24)$$

Accordingly, we do not advance to $E(t)$ scenario curves until μ is definition-consistent and $A(t)/g$ are clearly specified, avoiding non-computable narratives.

7 Simulation Results and Sensitivity Analysis

Provide visual data through plots. Discuss model robustness. Sensitivity analysis is crucial for winning Meritorious or Outstanding awards.

8 Discussion: Strengths and Weaknesses

8.1 Strengths

High predictive accuracy; adaptable to high-dimensional data.[3, 24]

8.2 Weaknesses

Computational complexity for large-scale matrices.

9 Conclusion

Summarize findings and provide actionable insights.

References

- [1] Nadia Alshahwan et al. Automated Unit Test Generation using Large Language Models at Meta. *arXiv preprint arXiv:2409.03093*, 2024. URL <https://arxiv.org/abs/2409.03093>. Accessed January 30, 2026.
- [2] Anthropic. Claude 2 Model Card, 2023. URL <https://www.anthropic.com/claude-2-model-card>. Accessed January 30, 2026.
- [3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128*, 2023. URL <https://arxiv.org/abs/2304.05128>. Accessed January 30, 2026.
- [4] Carl Benedikt Frey and Michael A. Osborne. The Future of Employment: How Susceptible are Jobs to Computerisation? Technical report, Oxford Martin School, University of Oxford, 2017. URL <https://www.oxfordmartin.ox.ac.uk/downloads/academic/future-of-employment.pdf>. Accessed January 30, 2026.
- [5] Google Research. Large Sequence Models for Software Development Activities. Google Research Blog, 2024. URL <https://research.google/blog/large-sequence-models-for-software-development-activities/>. Accessed January 30, 2026.
- [6] National Center for Education Statistics. Integrated Postsecondary Education Data System (IPEDS) 2018-19 Final Release, 2019. URL <https://nces.ed.gov/ipeds/>. Accessed January 30, 2026.
- [7] National Center for Education Statistics. CIP2020-SOC2018 Crosswalk, 2020. URL <https://nces.ed.gov/ipeds/cipcode/>. Accessed January 30, 2026.
- [8] O*NET Resource Center. O*NET Database 30.1, 2025. URL <https://www.onetcenter.org/database.html>. Accessed January 30, 2026.
- [9] OpenAI. GPT-4 Technical Report. Technical report, 2023. URL <https://cdn.openai.com/papers/gpt-4.pdf>. Accessed January 30, 2026.
- [10] U.S. Census Bureau. Post-Secondary Employment Outcomes (PSEO) Release R2025Q2, 2025. URL https://lehd.ces.census.gov/data/pseo_beta.html. Accessed January 30, 2026.

- [11] Kyle Wiggers. AI Models Still Struggle to Debug Software, Microsoft Study Shows. TechCrunch, 2025. URL <https://techcrunch.com/2025/04/10/ai-models-still-struggle-to-debug-software-microsoft-study-shows/>. Accessed January 30, 2026.

Appendices

Appendix A Python Implementation Code

```
1 def model_solver(data):  
2     # Core mathematical logic here  
3     return results
```

Report on Use of AI Tools

1. Dialogue Traceability

For each use, provide the exact **Prompt/Query** and the **raw AI Output** (or a representative excerpt). Omit full logs only for translation-only use; in that case, state the **tool name and version**.

- **Example—Prompt:** “Suggest an outline for the model section.” **Output (excerpt):** “1. State assumptions; 2. Define notation; 3. Derive main equations. . .”
- **Translation-only teams:** Declare tool name and version (e.g., DeepL 4.2, ChatGPT-4); no full prompt log required.

2. Phased Declaration

State in which phases AI was used, e.g.:

- **Initial conception / brainstorming**
- **Code correction / debugging**
- **Translation / copy-editing**

Mark each phase as used or not, and name the tool per phase.

3. Risk Verification

Describe how the team checked for **hallucinations** and **wrong citations**: e.g., manual fact-check of key claims, verification of references, cross-check of formulas and code. If only used for translation, state that no substantive technical content was delegated to AI.